Para utilizar a inteligência artificial na programação com eficiência e evitar que ela gere códigos com erros ou comportamento inesperado, é necessário adotar estratégias de **engenharia de prompt estruturada**, **restrições contratuais** e **auditoria crítica contínua** 1, 2.

1. Controle de Contexto e Prompts Focados

- **Não envie o arquivo inteiro:** Colar centenas de linhas de código faz com que o modelo disperse a atenção em trechos irrelevantes 3, 4. O ideal é isolar apenas a função ou módulo com falha e indicar os tipos/dependências imediatas 2, 3.
- **Defina o comportamento esperado vs. o real:** Explique com clareza o que o código deveria fazer e o que ele está fazendo no momento 3, 4.
- **Evite solicitações genéricas:** Pedir para a IA "deixar o código limpo" ou "refatorar" gera reescritas arbitrárias 4, 5. Defina exatamente o que busca (ex.: *extraia a lógica de validação para uma função auxiliar e substitua estruturas aninhadas por retornos antecipados*) 4, 5.
- **Divida tarefas complexas em etapas:** Em vez de pedir um sistema inteiro em um único prompt gigante, encadeie o desenvolvimento passo a passo (ex.: primeiro a rota base, depois a autenticação, depois o tratamento de erros) 2, 6.

2. Definição de Restrições Negativas e Critérios de Conclusão

- **Aplique restrições explícitas (*Do NOT constraints*):** Modelos de IA tendem a adicionar funcionalidades não solicitadas ou alterar assinaturas de funções se não forem restringidos 2, 7. Sempre inclua regras do tipo:
- *"Não adicione dependências externas adicionais."* 4, 7
- *"Não altere a assinatura nem o tipo de retorno da função existente."* 2, 7
- *"Não modifique arquivos fora do diretório X."* 7
- **Estabeleça o critério de conclusão (*Done criteria*):** Especifique o que determina que o código está pronto (ex.: lidar com entradas nulas, passar nos testes existentes e não gerar erros de tipagem) 2, 8.

3. Validação Crítica e Auditoria Humana (*Human-in-the-Loop*)

- **Nunca aceite ou copie código cegamente:** As IAs produzem código que aparenta ser plausível, mas que pode conter bugs de concorrência, alucinações de bibliotecas ou métodos inexistentes 9-11.
- **Execute e teste casos de borda (*Edge Cases*):** Teste ativamente o código gerado utilizando suítes de testes unitários e passe cenários limites (como listas vazias, dados duplicados, valores nulos ou limites de escala) 10, 12, 13.
- **Raciocínio e revisão de código:** Use a IA como revisora ou parceira de discussão, mas exija que ela explique o porquê das decisões ou aponte possíveis gargalos de complexidade assintótica (\\(O(N)\\)) antes de integrar a solução ao projeto principal 10, 13, 14.

4. Fortalecimento dos Fundamentos de Programação

- A capacidade de identificar onde a IA errou e como corrigi-la depende diretamente do domínio sobre **estruturas de dados, algoritmos e padrões de projeto** 15, 16.

- Desenvolvedores com fundamentos sólidos conseguem direcionar o modelo com precisão e auditar a arquitetura do código gerado, evitando depender passivamente da ferramenta 15, 16.

💡 **Quer que eu crie um modelo de template de prompt estruturado para você utilizar no seu dia a dia ao pedir refatoração ou correção de bugs para a IA?**